UMBC Parking Management System
Database Design Proposal

Mateo Jacome, Jason Chen

March 11, 2026

Contents

1 Executive Summary 2

1.1 Problem Statement 2

1.2 Parking Users 2

1.3 Scope of Implementation 2

1.4 Concrete Use Cases 4

2 Requirements & Conceptual Design 6

2.1 ER Diagram 6

2.2 Requirements and Constraints 7

2.3 Concurrency Scenario 8

2.4 Rationale 8

3 Logical Design 10

3.1 Schema List 10

3.2 Candidate Keys 11

3.3 Naming Conventions 11

3.4 Normalization 11

HW #1 Executive Summary

1.1 Problem Statement

Parking at a large commuter campus such as UMBC is inefficient, stressful, and difficult to enforce. Students frequently waste valuable time searching for open spaces. Visitors struggle to locate valid parking without clear guidance. Faculty and staff require access to restricted or premium areas. Meanwhile, enforcement is largely reactive and labor-intensive, relying on officers instead of automated validation mechanisms.

The core problem is the absence of a centralized, real-time database system that integrates parking permits, lot and spot management, vehicle data, reservations, sensor occupancy detection, and automated enforcement into a unified platform. Without such integration, parking operations lack efficiency, transparency, and scalability.

The proposed UMBC Parking Management System addresses this issue by implementing a PostgreSQL-backed database that manages parking as a real-time, data-driven ecosystem.

1.2 Parking Users

The system supports multiple categories of users:

- **Commuter Students** – Purchase student permits and park in designated student lots.
- **Faculty and Staff** – Hold faculty/staff permits and access restricted premium zones.
- **Visitors and Guest Lecturers** – Reserve temporary parking spaces online and pay per visit.

1.3 Scope of Implementation

This project will implement a PostgreSQL database system that includes:

1. **User and Vehicle Management**

- Store user profiles (students, faculty, visitors).
- Associate multiple vehicles with individual users.
- Store license plate information for validation.

2. **Permit Management**
    - Define permit types (student, faculty, visitor).
    - Assign permits to users.
    - Enforce expiration dates.
    - Restrict lot and zone access by permit type.
3. **Lot and Spot Management**
    - Define parking lots, zones, rows, and individual spots.
    - Track real-time occupancy per spot.
    - Associate spots with allowed permit types.
4. **Reservation System**
    - Allow visitors to reserve specific spots.
    - Prevent double-booking through constraints and transactions.
    - Lock spots upon confirmed payment.
5. **Sensor Integration**
    - Update occupancy when a ground sensor detects a vehicle.
    - Validate vehicle authorization against zone restrictions.
6. **Automated Ticketing**
    - Generate violations for unauthorized parking.
    - Store fine amounts and violation details.
    - Record notification events.
7. **Payments**
    - Record ticket payments.
    - Track outstanding balances.

8. **Reporting and Analytics**
    - Lot utilization rates.
    - Revenue reports.
    - Violation frequency reports.
    - Active and expired permit tracking.
    - Peak usage analysis.
9. **Concurrency Control**
    - Prevent race conditions during reservation.

This scope focuses specifically on the database design, implementation, constraints, indexing, realistic operations, and concurrency control. It does not include a full mobile or web application interface.

1.4 Concrete Use Cases

The database must support the following operations:

- Register a new user (student, faculty, or visitor).
- Add one or more vehicles to a user account.
- Purchase and assign a parking permit to a user.
- Define parking lots and individual parking spots.
- Associate permit types with allowed zones.
- Query real-time available parking spots in a specific lot.
- Create a visitor reservation for a specific date and time.
- Prevent double-booking of a parking spot.
- Update occupancy when a sensor detects a vehicle.
- Validate a vehicle's permit against the spot's allowed zone.

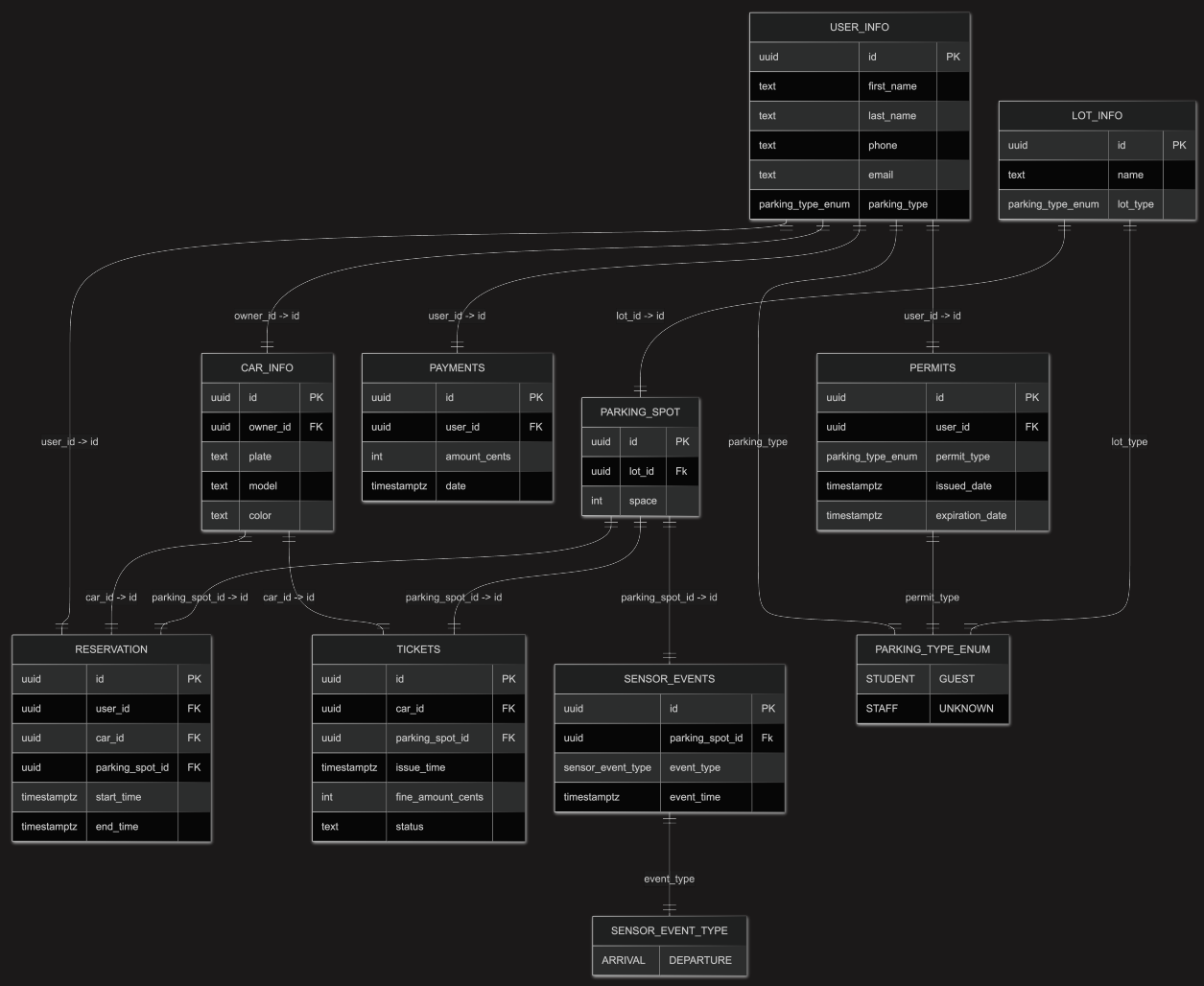
- Automatically generate a ticket for unauthorized parking.
- Record ticket payments and mark violations as resolved.
- Generate reports on lot occupancy over time.
- Generate revenue reports from permits, reservations, and fines.
- Detect expired permits and flag vehicles as invalid.
- Handle simultaneous reservation attempts without data corruption.

HW #2 Requirements & Conceptual Design

This section will more rigourously define the requirements, constraints, roles, as well as include an ER diagram.

2.1 ER Diagram

To see full image click on `img/er_diagram.png`



2.2 Requirements and Constraints

The following business rules define constraints for the UMBC Parking Management System. Each rule is labeled with how it is enforced: (A) through database table constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, or CHECK, or (B) through triggers, functions, or application logic.

Rule 1: Each user may register multiple vehicles, but each license plate must be unique across the system.

Enforcement: **(A)** A UNIQUE constraint is applied to the `car_info.plate` column.

Rule 2: Each parking spot within a lot must be uniquely identified by its lot and space number.

Enforcement: **(A)** A composite UNIQUE constraint is applied to (`lot_id`, `space`) in the `parking_spot` table.

Rule 3: A reservation must have a start time that occurs before its end time.

Enforcement: **(A)** A CHECK constraint ensures `start_time <end_time`.

Rule 4: A permit must expire after it is issued.

Enforcement: **(A)** A CHECK constraint ensures `expiration_date >issued_date`.

Rule 5: Monetary values for fines and payments must be positive.

Enforcement: **(A)** CHECK constraints ensure values such as `fine_amount_cents >0` and `amount_cents >0`.

Rule 6: A parking spot cannot be reserved by two users for overlapping time intervals.

Enforcement: **(B)** Implemented using a PostgreSQL trigger that looks at the time range derived from `start_time` and `end_time`.

Rule 7: Only vehicles with a valid and unexpired permit may park in permit-restricted lots.

Enforcement: **(B)** Implemented using a trigger or application-level validation that checks the vehicle's permit type and expiration date.

Rule 8: Visitor reservations must be confirmed before the space is considered locked.

Enforcement: **(B)** Application logic ensures reservations transition from `PENDING` to `CONFIRMED` only after successful payment.

Rule 9: Sensor events must record an arrival or departure event with a timestamp.

Enforcement: **(A)** Implemented using the `sensor_event_type` enum and a `NOT NULL` timestamp constraint.

Rule 10: If a vehicle parks without authorization (no valid permit or reservation), the system must generate a parking violation ticket.

Enforcement: **(B)** Implemented using a trigger that listens for arrival events and inserts a record into the tickets table when authorization fails.

2.3 Concurrency Scenario

A concurrency scenario occurs when two users attempt to reserve the same parking spot at nearly the same time. For example, User A and User B both attempt to reserve parking spot S in Lot 3 from 9:00 AM to 12:00 PM.

Without proper concurrency control, both reservations could be inserted into the database, creating a double booking. The system must guarantee that only one reservation succeeds.

To enforce this requirement, the database uses a PostgreSQL trigger that runs BEFORE insertion into the reservations table, that prevents overlapping time ranges for the same parking spot. When the first reservation transaction commits successfully, any subsequent reservation that overlaps with that time interval will automatically fail.

This approach ensures correctness even when multiple transactions occur simultaneously.

2.4 Rationale

The design separates long-term policy information from transactional records in order to improve flexibility and maintainability. Permit classifications (such as student, faculty, or visitor) represent policy rules that determine which parking areas are accessible to different groups. By separating permit types from individual permit records, the system allows parking policies to evolve independently from historical permit data.

Reservations are modeled using explicit start and end timestamps rather than a single date field so that the system can support flexible time windows. This design enables features such as hourly reservations, overlapping time checks, and historical analysis of parking usage patterns.

Sensor events are stored as individual event records instead of directly updating a parking spot's status. This event-driven model preserves a complete history of arrivals and departures, which is valuable for analytics, auditing, and debugging sensor behavior. A trigger or background process can interpret these events to update occupancy status, validate permits, or generate tickets when violations occur.

Finally, the schema models relationships such as vehicles belonging to users and reservations referencing parking spots using foreign keys. These relationships ensure referential integrity and make it possible to query the system efficiently for operations such as finding available parking, verifying permits, or generating enforcement reports.

HW #3 Logical Design

3.1 Schema List

Here `parking_type`, `lot_type`, `permit_type` and `event_type` are all SQL enum types. We have defined them in `db/migrations/01_enums.sql`

My ER Diagram takes normalization into account in order to avoid issues like redundancy and to ensure data consistency.

```
Lot_Info(id, name, lot_type)

User_Info(id, first_name, last_name, phone, email, parking_type)

Car_Info(id, plate, model, color, FK → User_Info(id))

Parking_Spot(id, space, FK → Lot_Info(id))

Reservation(id, start_time, end_time,
            FK → User_Info(id),
            FK → Car_Info(id),
            FK → Parking_Spot(id))

Sensor_Events(id, event_type, event_time,
            FK → Parking_Spot(id))

Permits(id, permit_type, issued_date, expiration_date,
        FK → User_Info(id))

Payments(id, amount_cents, date,
        FK → User_Info(id))

Tickets(id, issue_time, fine_amount_cents, status,
        FK → Car_Info(id),
        FK → Parking_Spot(id))
```

3.2 Candidate Keys

Each table uses a UUID (specifically, we're gonna use uuid v7 because it includes a time stamp so it's time sortable outa the box and will make indexing more efficient) primary key named `id`, chosen because it is globally unique and immutable. It's just better!!

Some tables also have additional candidate keys:

- `user_info`: `email` and `phone` are unique and could serve as candidate keys.
- `car_info`: `plate` is unique and could also serve as a candidate key.
- `parking_spot`: the combination `(lot_id, space)` is unique, but we use `id` as the primary key.

No composite primary keys were required because each entity can be uniquely identified using a single UUID.

3.3 Naming Conventions

Convention	Examples
Table names: `snake_case` nouns	`user_info`, `parking_spot`
Column names: `snake_case`, descriptive	`first_name`, `start_time`
Primary keys: always `id`	`id`
Foreign keys: `<table>_id`	`user_id`, `parking_spot_id`
Timestamps: descriptive	`start_time`, `issue_time`, `expiration_date`
Enums: clearly named	`parking_type_enum`, `sensor_event_type`

3.4 Normalization

Example 1 - User Info

If we had designed my `user_info` table to also include vehicle information it would've looked like the image below. This approach allows us to store a user with their car information in the same table but it runs into many issues.

For one, if a user wants to register more than one car, then we'd have to duplicate their user information just to add another vehicle which would lead to redundancy!

This approach would also make it so that users who own no cars have all those extra columns there for no reason.

Furthermore, if a user had for example 10 cars registered, and they wanted to change their name we'd then have to update 10 different rows! And if we forget to update all 10 rows then we'd have an anomaly because some of the rows will have the user's new name and some will have their old name. By separating the data into two tables we avoid all of these issues.

```
User_Info(id, first_name, last_name, phone, email, parking_type, plate,
↪ model, color)
```

Now the user data lives independently of the vehicle information and all the above issues are fixed. If the user wants to change their name, we have only have to update one row. A lot simpler.

```
User_Info(id, first_name, last_name, phone, email, parking_type)

Car_Info(id, plate, model, color, FK → User_Info(id))
```

Example 2 - Parking Spot

Another example of normalization can be seen in the design of the `parking_spot` table.

Suppose instead that the reservation table stored the parking space number and the lot information directly, as shown below.

```
Reservation(id, start_time, end_time, user_id, car_id, lot_name, space)
```

This approach would repeat the same parking lot and space information for every reservation

that occurs at that spot. If a parking spot is reserved hundreds of times throughout a semester, then the same lot name and space number would be duplicated in hundreds of rows.

This creates an update anomaly. If the name of a parking lot changes, then every reservation referencing that lot would need to be updated. If even one row is missed, the database would contain inconsistent data.

There is also a potential delete anomaly. If the last reservation associated with a particular parking space is deleted, the database would lose all information about that parking space.

To avoid these issues, parking spots are stored in their own table and referenced through a foreign key.

```
Parking_Spot(id, space, FK → Lot_Info(id))

Reservation(id, start_time, end_time,
            FK → User_Info(id),
            FK → Car_Info(id),
            FK → Parking_Spot(id))
```

With this design, each parking spot is stored exactly once in the `parking_spot` table. Reservations simply reference the spot using its UUID. This eliminates redundancy and ensures that changes to parking infrastructure only need to be made in a single location.